SENATI

Escuela de Tecnologías de la Información

GUÍA 04

Deep Learning con TensorFlow y Keras

Estudia el uso de las librerías TensorFlow y Keras

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Módulos y Paquetes para Machine Learning con Python
Módulo ocupacional	Especialista en Diseño y Desarrollo de Sistemas de Información con Machine Learning
Semestre / Semana	IV / Semana 3
Tarea de aprendizaje	Estudia el uso de las Librerías TensorFlow y Keras
Docente	Rembrandt Ubalde Enríquez
Estudiante	_____

Entorno de trabajo: Python + Visual Studio Code

Contenido

Contenido 2
Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 3
Paso 1. Crear la carpeta del proyecto 3
Paso 2. Crear y activar el entorno virtual (venv) 3
Paso 3. Instalar las librerías 4
Paso 4. Seleccionar el intérprete y verificar 4
Paso 5. Cómo ejecutar cada script de esta guía 5
Parte 2. Conceptos clave 6
2.1 ¿Qué es el Deep Learning? 6
2.2 Neurona biológica vs. neurona artificial 6
2.3 ¿Qué son TensorFlow y Keras? 7
2.4 Tipos de redes neuronales 8
2.5 Funciones de activación 8
2.6 Principales aplicaciones del Deep Learning 9
Parte 3. Prácticas con TensorFlow y Keras 11
Práctica 03. Tensores, variables y gradientes 11
Práctica 04. Tu primera red neuronal con Keras 12
Práctica 05. Red multicapa para clasificar flores Iris 13
Parte 4. Errores frecuentes y soluciones 14
Parte 5. Ejercicios propuestos 15
Preguntas de reflexión 15
Lista de verificación 15

(Si el índice aparece vacío, haz clic derecho sobre él y elige "Actualizar campo".)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Definir** el concepto de Deep Learning y compararlo con el funcionamiento de las neuronas biológicas.
- **Definir** las librerías TensorFlow y Keras, y reconocer para qué sirve cada una.
- **Identificar** los principales tipos de redes neuronales, sus funciones de activación y sus aplicaciones.
- **Construir, entrenar, evaluar y guardar** una red neuronal sencilla en Visual Studio Code.

Operación	Dónde se trabaja en esta guía
Define el concepto de Deep Learning	Parte 2 (secciones 2.1 y 2.2) y Práctica 01
Define la librería TensorFlow y Keras	Parte 2 (sección 2.3) y Prácticas 03 a 05
Identifica principales aplicaciones	Parte 2 (secciones 2.4 a 2.6)

Materiales

- Computadora con Windows 10/11 de 64 bits, 8 GB de RAM y conexión a internet (TensorFlow pesa unos 400 MB).
- Python **3.10, 3.11 o 3.12** y Visual Studio Code con las extensiones **Python** y **Pylance** de Microsoft.

Parte 1. Preparación del entorno en Visual Studio Code

Si ya instalaste Python y VS Code en las guías anteriores, no repitas la instalación: empieza en el **Paso 1** creando una carpeta nueva para esta guía y su propio entorno virtual.

Paso 1. Crear la carpeta del proyecto

1. Crea la carpeta **IA_Guia04_TensorFlow_Keras** en tu Escritorio o en Documentos.
2. Abre VS Code y elige **Archivo > Abrir carpeta…** (File > Open Folder). Selecciona la carpeta y pulsa **Seleccionar carpeta**.
3. Si aparece el mensaje "¿Confía en los autores de los archivos?", pulsa **Sí, confío en los autores**.
4. Abre la terminal integrada con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Debe abrirse en la ruta de tu carpeta.

> ⚠ **Atención:** Revisa que la ruta de la terminal termine en el nombre de tu carpeta (por ejemplo `...\IA_Guia04_TensorFlow_Keras>`). Si dice solo `C:\Users\tu_usuario>`, la carpeta no está abierta: repite el paso 2.

Paso 2. Crear y activar el entorno virtual (venv)

⌨ Terminal de VS Code (PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Al activarse, la línea de la terminal empieza con **(venv)**. Si aparece un error rojo de "ejecución de scripts deshabilitada", ejecuta lo siguiente, responde **S** y vuelve a activar:

⌨ Solo si aparece el error de scripts

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

⚠ **Atención:** Si al escribir `python` se abre la **Microsoft Store** o sale "Python no se encontró", Python no está instalado o no está en el PATH. Reinstálalo desde python.org marcando **"Add python.exe to PATH"**, desactiva en Windows *Configuración > Aplicaciones > Alias de ejecución de aplicaciones* los alias de python.exe y python3.exe, y reinicia VS Code.

Paso 3. Instalar las librerías

⌨ Terminal de VS Code (con (venv) activo)

```
python -m pip install --upgrade pip
pip install tensorflow scikit-learn matplotlib
```

⚠ **Atención:** TensorFlow solo se instala en versiones de Python que el proyecto soporta. Si pip responde **"No matching distribution found for tensorflow"**, revisa tu versión con `python --version`; si es más nueva que la soportada, instala Python 3.12 y vuelve a crear el venv.

💡 **Consejo:** Desde TensorFlow 2.16, **Keras 3 se instala automáticamente** junto con TensorFlow. Por eso no hace falta escribir `pip install keras`, y en el código se importa simplemente con `import keras`.

Paso 4. Seleccionar el intérprete y verificar

1. Presiona **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige la opción que dice **('venv': venv)**.
2. Comprueba en la barra de estado (abajo a la derecha) que aparezca **Python 3.x ('venv')**.
3. Verifica la instalación con el comando de abajo.

⌨ Verificación

```
python -c "import tensorflow as tf, keras; print(tf.__version__, keras.__version__)"
```

▶ Resultado esperado en la terminal

```
2.x.x 3.x.x
```

```
(pueden aparecer antes líneas informativas que empiezan con “I tensorflow/…” o “oneDNN
custom operations…”: son normales)
```

Paso 5. Cómo ejecutar cada script de esta guía

1. En el panel **Explorador** pulsa el ícono **Nuevo archivo** y escribe el nombre exacto indicado (por ejemplo `01_neurona.py`).
2. Copia el código, guarda con **Ctrl + S**.
3. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo en la terminal `python nombre_del_archivo.py`.
4. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

> 💡 **Consejo:** Cuando un script usa `plt.show()`, el gráfico se abre en una **ventana aparte**. El programa queda en pausa hasta que **cierres esa ventana**; recién entonces la terminal vuelve a estar disponible.

Parte 2. Conceptos clave

2.1 ¿Qué es el Deep Learning?

El **Deep Learning** (aprendizaje profundo) es una rama del Machine Learning que usa **redes neuronales artificiales con muchas capas** para aprender patrones directamente desde los datos. Se llama "profundo" porque los datos atraviesan varias capas ocultas, y cada una aprende una representación más abstracta que la anterior: en una foto, las primeras capas detectan bordes, las siguientes formas y las últimas objetos completos.

Inteligencia Artificial	Machine Learning	Deep Learning
Cualquier técnica que permite a una máquina imitar comportamiento inteligente.	Subconjunto de la IA: el sistema aprende reglas a partir de datos, sin programarlas a mano.	Subconjunto del ML: usa redes neuronales profundas; necesita más datos y más cómputo.
Ej.: un sistema experto con reglas SI-ENTONCES.	Ej.: árbol de decisión que predice si un cliente pagará.	Ej.: reconocimiento facial, traducción automática, ChatGPT.

2.2 Neurona biológica vs. neurona artificial

Las redes neuronales artificiales se inspiran (de forma simplificada) en el cerebro. La tabla compara sus partes:

Neurona biológica	Neurona artificial	¿Qué hace?
Dendritas	Entradas (x_1, x_2, x_3…)	Reciben la información.
Sinapsis	Pesos (w_1, w_2, w_3…)	Indican qué tan importante es cada entrada.
Cuerpo celular (soma)	Suma ponderada $z = \Sigma(x_i \cdot w_i) + b$	Combina toda la información recibida.
Umbral de disparo	Sesgo (b) + función de activación	Decide si la neurona "se activa" y con qué intensidad.
Axón	Salida (y)	Envía el resultado a la siguiente neurona o capa.
Aprende fortaleciendo conexiones	Aprende ajustando pesos (retropropagación)	Reduce el error en cada época de entrenamiento.

> 💡 **Consejo:** Diferencia importante: el cerebro humano tiene unos 86 000 millones de neuronas que trabajan en paralelo y consumen muy poca energía; una red artificial es un **modelo matemático** que solo se parece al cerebro en la idea general.

Práctica 01. Una neurona artificial con NumPy

Antes de usar TensorFlow, programaremos una neurona "a mano" para ver qué ocurre por dentro. La neurona decide si un estudiante aprueba según sus horas de estudio, asistencia y tareas.

01_neurona.py

```
# 01_neurona.py - Una neurona artificial "a mano" con NumPy
```

```
import numpy as np

# Entradas (como las dendritas): horas de estudio, asistencia (0 a 1), tareas entregadas
x = np.array([6.0, 0.9, 4.0])

# Pesos: importancia de cada entrada (la "fuerza" de cada sinapsis)
w = np.array([0.4, 2.0, 0.3])
b = -4.0                                    # sesgo (bias): umbral de activación

# 1. Suma ponderada (lo que hace el cuerpo de la neurona)
z = np.dot(x, w) + b
print("Suma ponderada z =", round(z, 3))

# 2. Función de activación sigmoide: convierte z en un valor entre 0 y 1
def sigmoide(z):
    return 1 / (1 + np.exp(-z))

salida = sigmoide(z)
print("Salida de la neurona (probabilidad de aprobar):", round(salida, 3))
print("Decisión:", "Aprueba" if salida >= 0.5 else "Desaprueba")

# 3. Probar con otro estudiante
x2 = np.array([1.0, 0.5, 1.0])
print("Estudiante 2 -> salida:", round(sigmoide(np.dot(x2, w) + b), 3))
```

▶ Resultado esperado en la terminal

```
Suma ponderada z = 1.4
Salida de la neurona (probabilidad de aprobar): 0.802
Decisión: Aprueba
Estudiante 2 -> salida: 0.091
```

Análisis: z = 6(0.4) + 0.9(2.0) + 4(0.3) − 4 = **1.4**. La sigmoide convierte 1.4 en **0.802** (80 %), por eso la neurona decide “Aprueba”. Para el estudiante 2, z = −2.3 y la salida baja a 0.091.

2.3 ¿Qué son TensorFlow y Keras?

	TensorFlow	Keras
Definición	Plataforma de código abierto creada por **Google** (2015) para cálculo numérico y aprendizaje automático a gran escala.	API de **alto nivel** para construir redes neuronales con pocas líneas de código. Desde 2019 es la interfaz oficial de TensorFlow.
Unidad básica	El **tensor**: arreglo de n dimensiones (escalar, vector, matriz…).	La **capa** (layer) y el **modelo** (Sequential o funcional).
Fortalezas	Derivadas automáticas, uso de GPU/TPU, despliegue en servidores, móviles (LiteRT) y web (TensorFlow.js).	Sintaxis simple y legible, ideal para aprender y prototipar rápido.
Importación	`import tensorflow as tf`	`import keras` y `from keras import layers`

Piensa en **TensorFlow como el motor** de un auto y en **Keras como el volante y los pedales**: Keras te permite manejar la red sin preocuparte por los detalles internos que TensorFlow resuelve.

2.4 Tipos de redes neuronales

Tipo	Cómo funciona	Aplicación típica	Capa en Keras
Perceptrón / Red densa (MLP)	Neuronas totalmente conectadas organizadas en capas.	Datos en tablas: aprobar crédito, predecir notas.	`layers.Dense`
Convolucional (CNN)	Filtros que recorren la imagen detectando bordes y formas.	Visión: rostros, placas, radiografías.	`layers.Conv2D`
Recurrente (RNN, LSTM, GRU)	Tiene "memoria" de los datos anteriores de una secuencia.	Series de tiempo, texto, voz.	`layers.LSTM`
Autoencoder	Comprime los datos y luego los reconstruye.	Detección de fraudes, quitar ruido.	`Dense` / `Conv2D`
Transformer	Mecanismo de **atención** que relaciona todas las palabras entre sí.	Traductores, chatbots (GPT, Claude).	`layers.MultiHeadAttention`
Generativa adversaria (GAN)	Dos redes compiten: una genera y otra evalúa.	Generar imágenes sintéticas.	Modelos combinados

2.5 Funciones de activación

La **función de activación** transforma la suma ponderada z en la salida de la neurona. Sin ella, por más capas que tenga, la red solo podría aprender relaciones en línea recta.

Función	Fórmula / rango	¿Dónde se usa?
Sigmoide	$1 / (1 + e^{-z})$ → entre 0 y 1	Salida de clasificación **binaria** (sí/no).
Tanh	Tangente hiperbólica → entre −1 y 1	Capas ocultas de redes recurrentes.
ReLU	max(0, z) → 0 o positivo	La más usada en **capas ocultas**: rápida y simple.
Leaky ReLU	z si z > 0; 0.1·z si no	Variante de ReLU que evita neuronas "muertas".
Softmax	$e^{zi} / \Sigma e^{z}$ → probabilidades que suman 1	Salida de clasificación **multiclase** (3 o más clases).
Lineal	z (sin cambios)	Salida de **regresión** (predecir un número).

Práctica 02. Graficar las funciones de activación

02_activaciones.py

```
# 02_activaciones.py - Principales funciones de activación
import numpy as np
import matplotlib.pyplot as plt

z = np.linspace(-5, 5, 200)

sigmoide = 1 / (1 + np.exp(-z))
tanh = np.tanh(z)
```

```
relu = np.maximum(0, z)
leaky_relu = np.where(z > 0, z, 0.1 * z)

# Softmax: convierte puntajes en probabilidades que suman 1
puntajes = np.array([2.0, 1.0, 0.1])
softmax = np.exp(puntajes) / np.exp(puntajes).sum()
print("Puntajes:", puntajes)
print("Softmax :", softmax.round(3), "| suma =", softmax.sum().round(3))

valores = np.array([-2.0, 0.0, 2.0])
print("Para z =", valores)
print("  sigmoide:", (1 / (1 + np.exp(-valores))).round(3))
print("  tanh    :", np.tanh(valores).round(3))
print("  relu    :", np.maximum(0, valores))

fig, ejes = plt.subplots(1, 4, figsize=(13, 3))
for ax, y, titulo in zip(ejes, [sigmoide, tanh, relu, leaky_relu],
                         ["Sigmoide (0 a 1)", "Tanh (-1 a 1)", "ReLU", "Leaky ReLU"]):
    ax.plot(z, y, linewidth=2)
    ax.axhline(0, color="gray", linewidth=0.7)
    ax.axvline(0, color="gray", linewidth=0.7)
    ax.set_title(titulo)
    ax.grid(alpha=0.3)
fig.tight_layout()
plt.savefig("02_activaciones.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 02_activaciones.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Puntajes: [2.  1.  0.1]
Softmax : [0.659 0.242 0.099] | suma = 1.0
Para z = [-2.  0.  2.]
  sigmoide: [0.119 0.5   0.881]
  tanh    : [-0.964  0.     0.964]
  relu    : [0. 0. 2.]
Imagen guardada: 02_activaciones.png
```

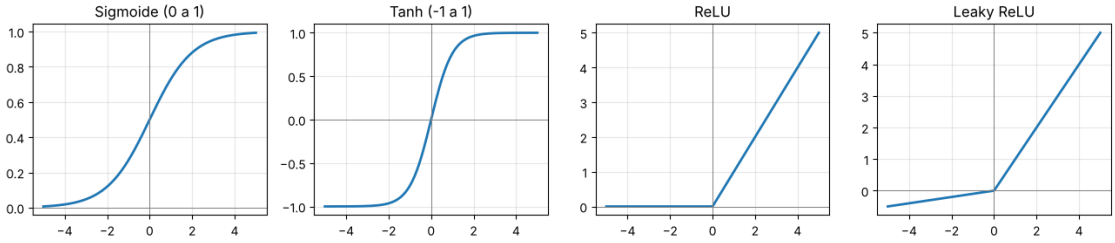


Figura 1. Ventana que se abre al ejecutar 02_activaciones.py

Observa que Softmax convirtió los puntajes [2, 1, 0.1] en probabilidades **0.659 + 0.242 + 0.099 = 1**: la red "elige" la clase con mayor probabilidad.

2.6 Principales aplicaciones del Deep Learning

Área	Ejemplos reales
Visión por computadora	Reconocimiento facial en celulares, lectura de placas vehiculares, control de calidad en fábricas.

Área	Ejemplos reales
Procesamiento de lenguaje (PLN)	Traductores, asistentes como ChatGPT o Claude, análisis de opiniones de clientes.
Voz y audio	Asistentes de voz, subtítulos automáticos, transcripción de reuniones.
Salud	Detección de tumores en radiografías y apoyo al diagnóstico.
Finanzas y comercio	Detección de fraudes con tarjetas, sistemas de recomendación (Netflix, Spotify).
Transporte y agro	Vehículos autónomos, drones que detectan plagas en cultivos.

Parte 3. Prácticas con TensorFlow y Keras

> ⚠ **Atención:** En las prácticas 03 a 05 los valores decimales de pérdida y exactitud pueden variar ligeramente según el equipo y la versión instalada. Lo importante es que la **pérdida baje** y la **exactitud suba** con el entrenamiento.

Práctica 03. Tensores, variables y gradientes

Un **tensor** es la estructura de datos de TensorFlow: un arreglo de n dimensiones, muy parecido a un arreglo de NumPy. `GradientTape` calcula derivadas automáticamente; así es como una red sabe hacia dónde ajustar sus pesos.

03_tensores.py

```
# 03_tensores.py - Primeros pasos con TensorFlow: tensores y gradientes
import tensorflow as tf

print("Versión de TensorFlow:", tf.__version__)

# 1. Un tensor es un arreglo de n dimensiones (como un arreglo de NumPy)
vector = tf.constant([1.0, 2.0, 3.0])
print("Vector:", vector.numpy(), "| forma:", vector.shape, "| tipo:", vector.dtype)

a = tf.constant([[1.0, 2.0], [3.0, 4.0]])
b = tf.constant([[5.0, 6.0], [7.0, 8.0]])

# 2. Operaciones con tensores
print("Suma:\n", (a + b).numpy())
print("Producto matricial:\n", tf.matmul(a, b).numpy())
print("Media de a:", tf.reduce_mean(a).numpy())

# 3. Variables: valores que la red modifica al aprender (pesos)
peso = tf.Variable(0.5)
peso.assign_add(0.25)
print("Peso actualizado:", peso.numpy())

# 4. GradientTape calcula derivadas automáticamente (base del aprendizaje)
x = tf.Variable(3.0)
with tf.GradientTape() as cinta:
    y = x ** 2 + 2 * x            # y = x² + 2x
derivada = cinta.gradient(y, x)   # dy/dx = 2x + 2
print("y en x=3:", y.numpy(), "| derivada:", derivada.numpy())
```

▶ Resultado esperado en la terminal

```
Versión de TensorFlow: 2.x.x
Vector: [1. 2. 3.] | forma: (3,) | tipo: <dtype: 'float32'>
Suma:
 [[ 6.  8.]
 [10. 12.]]
Producto matricial:
 [[19. 22.]
 [43. 50.]]
Media de a: 2.5
Peso actualizado: 0.75
y en x=3: 15.0 | derivada: 8.0
```

Comprobación: la derivada de $y = x^2 + 2x$ es $2x + 2$; en $x = 3$ vale $2(3) + 2 = \mathbf{8}$, igual que el resultado de TensorFlow.

Práctica 04. Tu primera red neuronal con Keras

Le daremos a la red 11 ejemplos de la regla **y = 2x + 1** sin decirle la fórmula. Después del entrenamiento revisaremos si la descubrió.

04_primera_red.py

```
# 04_primera_red.py - La red más simple: aprender la regla y = 2x + 1
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(42)          # resultados repetibles

# Datos de entrenamiento: la red NO conoce la fórmula, solo ve ejemplos
X = np.arange(-5, 6, dtype="float32")    # -5, -4, ..., 5
y = 2 * X + 1
print("x:", X)
print("y:", y)

# Modelo: 1 entrada -> 1 neurona (sin activación = salida lineal)
modelo = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(1),
])
modelo.compile(optimizer=keras.optimizers.SGD(learning_rate=0.01), loss="mse")

# Entrenamiento: 500 épocas (vueltas completas sobre los datos)
historial = modelo.fit(X, y, epochs=500, verbose=0)
for epoca in [1, 100, 250, 500]:
    print(f"Época {epoca:3}: pérdida (MSE) = {historial.history['loss'][epoca -
1]:.4f}")

# ¿Qué aprendió?
w, b = modelo.layers[0].get_weights()
print(f"Peso aprendido: {w[0][0]:.3f} (ideal 2) | Sesgo aprendido: {b[0]:.3f} (ideal
1)")
prediccion = modelo.predict(np.array([[10.0]]), verbose=0)
print(f"Predicción para x = 10: {prediccion[0][0]:.2f} (valor real 21)")
```

▶ Resultado aproximado (los decimales varían en cada equipo)

```
x: [-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.  5.]
y: [-9. -7. -5. -3. -1.  1.  3.  5.  7.  9. 11.]
Época   1: pérdida (MSE) = xx.xxxx     (valor alto: al inicio la red se equivoca mucho)
Época 100: pérdida (MSE) = 0.0xxx
Época 250: pérdida (MSE) = 0.000x
Época 500: pérdida (MSE) = 0.0000      (casi cero: aprendió la regla)
Peso aprendido: 2.000 (ideal 2) | Sesgo aprendido: 1.000 (ideal 1)
Predicción para x = 10: 21.00 (valor real 21)
```

Concepto	Significado en este ejemplo
`keras.Sequential`	Modelo formado por capas apiladas una detrás de otra.

Concepto	Significado en este ejemplo
`layers.Dense(1)`	Una capa con 1 neurona conectada a todas las entradas.
`compile`	Configura el **optimizador** (cómo ajusta los pesos) y la **pérdida** (cómo mide el error).
`fit` / épocas	Entrena la red; una época es una vuelta completa por todos los datos.
MSE	Error cuadrático medio: promedio de los errores al cuadrado. Mientras más cerca de 0, mejor.
`predict`	Usa la red ya entrenada para calcular nuevos valores.

Práctica 05. Red multicapa para clasificar flores Iris

Ahora construiremos una red "profunda" con **dos capas ocultas** que clasifica 3 especies de flores (setosa, versicolor, virginica) a partir de 4 medidas. Es el mismo dataset que usaste con Scikit-learn en la Guía 02, así podrás comparar resultados.

05_red_iris.py

```
# 05_red_iris.py - Red neuronal multicapa con Keras para clasificar flores Iris
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras import layers
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

keras.utils.set_random_seed(42)

# 1. Datos: 150 flores, 4 medidas, 3 especies
iris = load_iris()
X, y = iris.data, iris.target
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

escalador = StandardScaler()                # pone todas las medidas en la misma escala
X_ent = escalador.fit_transform(X_ent)
X_pru = escalador.transform(X_pru)
print("Entrenamiento:", X_ent.shape, "| Prueba:", X_pru.shape)

# 2. Arquitectura: 4 entradas -> 10 (ReLU) -> 8 (ReLU) -> 3 (Softmax)
modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    layers.Dense(10, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax"),   # una probabilidad por especie
])
modelo.summary()

# 3. Compilar: optimizador, función de pérdida y métrica
modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

# 4. Entrenar
historial = modelo.fit(X_ent, y_ent, epochs=100, batch_size=8, validation_split=0.1,
verbose=0)
print("Exactitud final en entrenamiento:", round(historial.history["accuracy"][-1], 3))
```

```
# 5. Evaluar con datos que la red nunca vio
perdida, exactitud = modelo.evaluate(X_pru, y_pru, verbose=0)
print(f"Exactitud en prueba: {exactitud:.3f}")

# 6. Predecir una flor nueva
flor = escalador.transform([[5.1, 3.5, 1.4, 0.2]])
prob = modelo.predict(flor, verbose=0)[0]
print("Probabilidades:", prob.round(3))
print("Especie predicha:", iris.target_names[np.argmax(prob)])

# 7. Guardar el modelo entrenado y la curva de aprendizaje
modelo.save("modelo_iris.keras")
print("Modelo guardado en modelo_iris.keras")

plt.plot(historial.history["accuracy"], label="Entrenamiento")
plt.plot(historial.history["val_accuracy"], label="Validación")
plt.title("Curva de aprendizaje")
plt.xlabel("Época")
plt.ylabel("Exactitud")
plt.legend()
plt.savefig("curva_aprendizaje.png", dpi=100, bbox_inches="tight")
plt.show()
```

▶ **Resultado aproximado (los decimales varían en cada equipo)**

```
Entrenamiento: (120, 4) | Prueba: (30, 4)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	50
dense_1 (Dense)	(None, 8)	88
dense_2 (Dense)	(None, 3)	27

```
 Total params: 165 (660.00 B)
 Trainable params: 165 (660.00 B)
 Non-trainable params: 0 (0.00 B)
Exactitud final en entrenamiento: 0.9xx
Exactitud en prueba: 0.9xx      (normalmente entre 0.90 y 1.00)
Probabilidades: [0.99x 0.00x 0.000]
Especie predicha: setosa
Modelo guardado en modelo_iris.keras
```

¿De dónde salen los 165 parámetros? Cada capa Dense tiene (entradas × neuronas) pesos + 1 sesgo por neurona: 4×10+10 = **50**, 10×8+8 = **88** y 8×3+3 = **27**. Total = 165 valores que la red ajusta al aprender.

Al final se abre una ventana con la **curva de aprendizaje**: las dos líneas deben subir y estabilizarse cerca de 1.0. Si la línea de validación queda muy por debajo de la de entrenamiento, el modelo está **memorizando** (sobreajuste).

> 💡 **Consejo:** Para usar el modelo guardado en otro programa, sin volver a entrenar: `modelo = keras.models.load_model("modelo_iris.keras")`.

Parte 4. Errores frecuentes y soluciones

Error o síntoma	Causa	Solución
`No matching distribution found for tensorflow`	Versión de Python no soportada por TensorFlow.	Instala Python 3.12, borra la carpeta venv y créala de nuevo.
`ModuleNotFoundError: No module named 'tensorflow'`	No está activo el venv o el intérprete no es el del venv.	Activa con `.\venv\Scripts\Activate.ps1` y elige el intérprete **('venv')**.
`DLL load failed while importing _pywrap_tensorflow`	Falta Microsoft Visual C++ Redistributable.	Descarga e instala "Microsoft Visual C++ Redistributable" (x64) y reinicia.
Mensajes con `oneDNN custom operations` o `I tensorflow/...`	Avisos informativos, no son errores.	Ignóralos; el programa sigue funcionando.
`ValueError: Input 0 ... expected shape=(None, 4)`	Los datos no tienen la forma que espera la capa de entrada.	Revisa `X.shape` y el `shape` de `keras.Input`.
La pérdida aparece como `nan`	Tasa de aprendizaje demasiado alta.	Reduce `learning_rate` (por ejemplo a 0.001).
Python se abre en la Microsoft Store	Python no instalado o no está en el PATH.	Ver el aviso del Paso 2 de la Parte 1.

Parte 5. Ejercicios propuestos

1. En `01_neurona.py`, cambia los pesos para que la asistencia sea lo **más importante**. Prueba con tres estudiantes distintos y explica los resultados.
2. En `02_activaciones.py`, agrega una quinta gráfica con la función **lineal** (y = z).
3. En `04_primera_red.py`, cambia los datos para que la red aprenda **y = 3x – 2**. ¿Qué peso y sesgo aprendió?
4. En `05_red_iris.py`, cambia la capa de 10 neuronas por una de **4 neuronas** y luego por una de **32**. Anota la exactitud en prueba de cada caso en una tabla.
5. Cambia `epochs=100` por `epochs=20`. ¿Qué pasa con la exactitud y con la curva de aprendizaje?
6. **Reto:** escribe un script `06_cargar_modelo.py` que cargue `modelo_iris.keras` y prediga la especie de la flor [6.7, 3.0, 5.2, 2.3]. (Recuerda que también debes escalar los datos.)

Preguntas de reflexión

1. ¿En qué se parecen y en qué se diferencian una neurona biológica y una artificial?
2. ¿Por qué la capa de salida de la red Iris usa Softmax y no Sigmoide?
3. ¿Qué tipo de red neuronal usarías para reconocer placas vehiculares? ¿Y para predecir la demanda de un producto mes a mes?

Lista de verificación

☐ Creé la carpeta del proyecto y el entorno virtual, y aparece **(venv)** en la terminal.

- ☐ Instalé TensorFlow y la verificación muestra las versiones de TensorFlow y Keras.
- ☐ Puedo explicar qué es el Deep Learning y compararlo con las neuronas biológicas.
- ☐ Ejecuté `01_neurona.py` y entiendo la suma ponderada y la activación.
- ☐ Grafiqué las funciones de activación y sé cuándo usar ReLU, Sigmoide y Softmax.
- ☐ Entrené mi primera red con Keras y aprendió la regla y = 2x + 1.
- ☐ Entrené la red Iris, obtuve su exactitud y guardé el modelo `modelo_iris.keras`.
- ☐ Resolví al menos 4 ejercicios propuestos.